

Лабораторная работа «Сетевые сокеты»

Абстракция сокетов была введена в BSD Unix (Berkley Software Distribution), поэтому сокеты часто называют сокетами BSD.

Сокет представляет собой абстракцию конечной точки взаимодействия.

```
#include <sys/types.h>
```

```
#include <sys/socket.h>
```

```
Sock_fd = socket(int socket_family, int socket_type, int protocol);
```

socket() создаёт, так называемую, конечную точку соединения (взаимодействия, коммуникации) и возвращает файловый дескриптор.

В Линукс определен унифицированный интерфейс между пользовательскими процессами и стеком сетевых протоколов в ядре.

Модули протоколов группируются по *семействам протоколов*: **AF_UNIX**, **AF_INET**, **AF_IPX** и **AF_PACKET**,

и *типам сокетов*: **SOCK_STREAM** или **SOCK_DGRAM**. Более подробная информация о семействах и типах приведена в man pages **socket(2)**.

Первый задает домен соединения или семейство адресов (address family - AF). Семейства описаны в <sys/socket.h>. В настоящее время определены такие форматы:

- AF_UNIX – сокеты в файловом пространстве имен.
- AF_INET – сетевые сокеты по протоколу TCP/IP для сетей IPv4.
- AF_INET6 - сетевые сокеты по протоколу TCP/IP для сетей IPv6.
- AF_UNSPEC – не специфицированное семейство

и некоторые другие.

Второй параметр функции **socket()** - **тип** сокета (*type*). В настоящее время определены следующие типы:

SOCK_STREAM

Обеспечивает создание двусторонних, надёжных потоков байтов на основе установления соединения. Может также поддерживаться механизм внепоточных данных.

SOCK_DGRAM

Поддерживает дейтаграммы (ненадежные сообщения с ограниченной длиной без установки соединения).

SOCK_SEQPACKET

Обеспечивает работу последовательного двустороннего канала для передачи дейтаграмм на основе соединений; дейтаграммы имеют постоянный размер; от получателя требуется за один раз прочитать целый пакет.

SOCK_RAW

Обеспечивает прямой доступ к сетевому протоколу.

SOCK_RDM

Обеспечивает надежную доставку дейтаграмм без гарантии, что они будут расположены по порядку.

SOCK_PACKET

Этот тип устарел и не должен использоваться в новых программах.

Некоторые типы сокетов могут быть не реализованы во всех семействах протоколов. Начиная с Linux 2.6.27, аргумент *type* предназначается для двух вещей: кроме определения

типа сокета, для изменения поведения **socket()** он может содержать побитовую сумму любых следующих значений:

SOCK_NONBLOCK - устанавливает флаг состояния файла **O_NONBLOCK** для нового открытого файлового дескриптора. Использование данного флага заменяет дополнительные вызовы **fcntl(2)** для достижения того же результата.

SOCK_CLOEXEC - устанавливает флаг close-on-exec (**FD_CLOEXEC**) для нового открытого файлового дескриптора. Смотрите описание флага **O_CLOEXEC** в **open(2)** для того, чтобы узнать, как это может пригодиться.

Since Linux 2.6.27, the *type* argument serves a second purpose: in addition to specifying a socket type, it may include the bitwise OR of any of the following values, to modify the behavior of **socket()**:

SOCK_NONBLOCK

Set the **O_NONBLOCK** file status flag on the new open file description. Using this flag saves extra calls to **fcntl(2)** to achieve the same result.

SOCK_CLOEXEC

Set the close-on-exec (**FD_CLOEXEC**) flag on the new file descriptor. See the description of the **O_CLOEXEC** flag in **open(2)** for reasons why this may be useful.

В третьем параметре - ***protocol*** задаётся определённый протокол, используемый с сокетом. Обычно, только единственный протокол существует для поддержки определённого типа сокета с заданным семейством протоколов. В этом случае для параметра *protocol* можно указать 0. Однако, может существовать несколько протоколов.

Протокол зависит от типа сокета, по которому устанавливается соединение (см. **protocols(5)**). Смотрите **getprotoent(3)**, где описано, как соотносить имена протоколов с их номерами.

Название	Назначение	Справочная страница
AF_UNIX, AF_LOCAL	Локальное соединение	unix(7)
AF_INET	Протоколы Интернет IPv4	ip(7)
AF_INET6	Протоколы Интернет IPv6	ipv6(7)
AF_IPX	Протоколы Novell IPX	
AF_NETLINK	Устройство для взаимодействия с ядром	netlink(7)
AF_X25	Протокол ITU-T X.25/ISO-8208	x25(7)
AF_AX25	Протокол любительского радио AX.25	
AF_ATMPVC	Доступ к низкоуровневым PVC в ATM	
AF_APPLETALK	AppleTalk	ddp(7)
AF_PACKET	Низкоуровневый пакетный интерфейс	packet(7)
AF_ALG	Интерфейс к ядерному крипто-API	

Типы сокетов

Сокеты типа **SOCK_STREAM** являются соединениями полнодуплексных байтовых потоков. Они не сохраняют границы записей. Поточковый сокет должен быть в состоянии *соединения* перед тем, как из него можно будет отсылать данные или принимать их. Соединение с другим сокетом создается с помощью системного вызова **connect(2)**. После соединения данные можно передавать с помощью системных вызовов **read(2)** и **write(2)** или одного из вариантов системных вызовов **send(2)** и **recv(2)**. Когда сеанс закончен, выполняется команда **close(2)**. Внепоточные данные могут передаваться, как описано в **send(2)**, и приниматься, как описано в **recv(2)**.

Протоколы связи, которые реализуют **SOCK_STREAM**, выполняются так, чтобы данные не были потеряны или дублированы. Если часть данных, для которых имеется место в буфере протокола, не может быть передана за определённое время, соединение считается разорванным. Когда в сокете включен флаг **SO_KEEPALIVE**, протокол каким-либо способом проверяет, не отключена ли ещё другая сторона.

*Если процесс посылает или принимает данные, пользуясь «разорванным» потоком, ему выдаётся сигнал **SIGPIPE**; это приводит к тому, что процессы, не обрабатывающие этот сигнал, завершаются.*

Сокеты **SOCK_SEQPACKET** используют те же самые системные вызовы, что и сокеты **SOCK_STREAM**. Единственное отличие в том, что вызовы **read(2)** возвращают только запрошенное количество данных, а остальные данные пришедшего пакета будут отброшены. Границы сообщений во входящих дейтаграммах сохраняются.

Сокеты SOCK_DGRAM и **SOCK_RAW** позволяют посылать дейтаграммы принимающей стороне, заданной при вызове **sendto(2)**. Дейтаграммы обычно принимаются с помощью

вызова **recvfrom(2)**, который возвращает следующую дейтаграмму с соответствующим обратным адресом.

Тип **SOCK_PACKET** считается устаревшим типом сокета; он позволяет получать необработанные пакеты прямо от драйвера устройства. Используйте вместо него **packet(7)**. Системный вызов **fcntl(2)** с аргументом **F_SETOWN** может использоваться для задания группы процессов, которая будет получать сигнал **SIGURG**, когда прибывают внепоточные данные, или сигнал **SIGPIPE**, когда соединение типа **SOCK_STREAM** неожиданно обрывается. Этот вызов также можно использовать, чтобы задать процесс или группу процессов, которые получают асинхронные уведомления о событиях ввода-вывода с помощью **SIGIO**. Использование **F_SETOWN** эквивалентно использованию вызова **ioctl(2)** с аргументом **FIOSETOWN** или **SIOCSPGRP**.

Когда сеть сообщает модулю протокола об ошибке (например, в случае IP, используя ICMP-сообщение), то для сокета устанавливается флаг ожидающей ошибки. Следующая операция этого сокета вернёт код ожидающей ошибки. Некоторые протоколы позволяют организовывать очередь ошибок в соке для получения подробной информации об ошибке; см. **IP_RECVERR** в **ip(7)**.

Операции сокетов контролируются их параметрами *options*. Эти параметры описаны в `<sys/socket.h>`. Вызовы **setsockopt(2)** и **getsockopt(2)** используются, чтобы установить и получить необходимые параметры соответственно.

ВОЗВРАЩАЕМОЕ ЗНАЧЕНИЕ

В случае успешного выполнения возвращается **дескриптор**, ссылающийся на сокет. В случае ошибки возвращается -1, а значение *errno* устанавливается соответствующим образом.

ОШИБКИ

EACCES

Нет прав на создание сокета указанного типа и/или протокола

EAFNOSUPPORT

Реализация не поддерживает указанное семейство адресов.

EINVAL

Неизвестный протокол или недоступное семейство протоколов.

EINVAL

Неверные флаги в *type*.

EMFILE

Было достигнуто ограничение по количеству открытых файловых дескрипторов на процесс.

ENFILE

Достигнуто максимальное количество открытых файлов в системе.

ENOBUFS или ENOMEM

Недостаточно памяти для создания сокета. Сокет не может быть создан, пока не будет освобождено достаточное количество ресурсов.

EPROTONOSUPPORT

Тип протокола или указанный протокол не поддерживаются в этом домене. Другие ошибки могут быть созданы модулями протоколов более низкого уровня.

СООТВЕТСТВИЕ СТАНДАРТАМ

POSIX.1-2001, POSIX.1-2008, 4.4BSD.

Флаги **SOCK_NONBLOCK** и **SOCK_CLOEXEC** есть только в Linux.

Вызов **socket()** появился в 4.2BSD. Обычно он переносим в/из не-BSD систем на уровне сокетов BSD (включая варианты System V).

Структуры адреса сокета

Каждый сокетный домен имеет свой формат сокетных адресов, выраженный в отдельной адресной структуре. Каждая из этих структур начинается с целочисленного поля «семейства» (с типом *sa_family_t*), в котором указывается тип адресной структуры. Это позволяет различным системным вызовам (например, **connect(2)**, **bind(2)**, **accept(2)**, **getsockname(2)**, **getpeername(2)**), которые являются общими для всех сокетов, определить домен конкретного сокетного адреса.

Для передачи сокетного адреса любого типа через программный интерфейс сокетов служит тип **struct sockaddr**. Целью данного типа является приведение типов сокетных адресов определённого домена к «общему» типу, что позволяет избежать предупреждений компилятора о несовпадении типов в вызовах API сокетов.

Функции, определенные на сокетах

Эти функции используются пользовательскими процессами для отправки или приёма пакетов и выполнения других операций над сокетами. Более подробная информация приведена в соответствующих справочных страницах.

Вызовы:

- **socket(2)** создаёт сокет,
 - **connect(2)** соединяет сокет с удалённым сокетным адресом,
 - **bind(2)** привязывает сокет к локальному адресу,
 - **listen(2)** сообщает сокету, что должны приниматься новые соединения,
 - **accept(2)** используется для получения нового сокета для нового входящего соединения.
 - **send(2)**, **sendto(2)** и **sendmsg(2)** отправляют данные в сокет,
 - **recv(2)**, **recvfrom(2)** и **recvmsg(2)** принимают данные из сокета.
 - для чтения и записи данных могут использоваться стандартные операции ввода-вывода: **write(2)**, **writev(2)**, **sendfile(2)**, **read(2)** и **readv(2)**.
 - **poll(2)** и **select(2)** ожидают поступления данных или готовятся к передаче данных.
- Пользователь может подождать наступления различных событий через poll() или select().**

Вызов **getsockname(2)** возвращает адрес локального сокета, а **getpeername(2)** возвращает адрес удалённого сокета. Вызовы **getsockopt(2)** и **setsockopt(2)** используются для установки или считывания параметров протокола или уровня сокетов. Вызов **ioctl(2)** может быть использован для установки или чтения некоторых других параметров.

Вызов **close(2)** используется для закрытия сокета. Вызов **shutdown(2)** закрывает части полнодуплексного сокетного соединения.

Перемещение (seeking), или вызовы **pread(2)** и **pwrite(2)** с ненулевой позицией, для сокетов не поддерживается.

Для **сокетов** возможно создание **неблокирующего ввода/вывода** путём установки в файловый дескриптор сокета флага **O_NONBLOCK** с помощью вызова **fcntl(2)**. При этом все блокировавшие раньше операции, будут возвращать **EAGAIN** (операция должна быть повторена позднее); **connect(2)** возвратит ошибку **EINPROGRESS**.

События ввода-вывода		
Событие	Флаг poll	Когда происходит
Чтение	POLLIN	Поступили новые данные
Чтение	POLLIN	Установка соединения выполнена (для сокетов, ориентированных на соединение)
Чтение	POLLHUP	Другая сторона инициировала запрос на разъединение
Чтение	POLLHUP	Соединение разорвано (только для протоколов, ориентированных на соединение). Если производится запись в сокет, то также посылается сигнал SIGPIPE
Запись	POLLOUT	Сокет имеет достаточно места в буфере отправки для записи в него новых данных
Чтение/Запись	POLLIN POLLOUT	Исходящий вызов connect(2) завершён
Чтение/Запись	POLLERR	Произошла асинхронная ошибка
Чтение/Запись	POLLHUP	Другая сторона закрыла (shut down) одно направление
Исключение	POLLPRI	Пришли неотложные данные. При этом посылается сигнал SIGURG

В ядре существует возможность информировать приложение о событиях с помощью сигнала **SIGIO**, что является альтернативой вызовам **poll(2)** и **select(2)**. Для этого необходимо установить с помощью **fcntl(2)** в файловом дескрипторе сокета флаг **O_ASYNC**, а также назначить с помощью **sigaction(2)** корректный обработчик сигнала **SIGIO**.

Адреса сокетов

Функция **socket()** создает "безымянный" сокет, т.е. не связанный ни с локальным адресом, ни с номером порта. Прежде чем передавать данные через сокет, его необходимо связать с адресом в выбранном домене (эту процедуру называют именованием сокета). Иногда связывание

осуществляется неявно (внутри функций **connect** и **accept**), но выполнять его необходимо во всех случаях. Вид адреса зависит от выбранного вами домена. В Unix-доме́не это текстовая строка - имя файла, через который происходит обмен данными. В Internet-доме́не адрес задаётся комбинацией IP-адреса и 16-битного номера порта. IP-адрес определяет хост в сети, а порт - конкретный сокет на этом хосте. Протоколы TCP и UDP используют различные пространства портов.

Для явного связывания сокета с некоторым адресом используется функция **bind**. Её прототип имеет вид:

```
#include <sys/types.h>
#include <sys/socket.h>
int bind(int sockfd, struct sockaddr *addr, int addrlen);
```

В качестве первого параметра передаётся дескриптор сокета, который привязывается к заданному адресу. Второй параметр – **addr** - содержит указатель на структуру struct sockaddr, а третий - длину этой структуры.

```
struct sockaddr
{
    unsigned short sa_family; // Семейство адресов, AF_xxx
    char sa_data[14]; // 14 байтов для хранения адреса
};
```

Структура определяет адрес в самом общем виде. Для сетевого взаимодействия определена другая структура **sockaddr_in**:

```
struct sockaddr_in
{
    short int sin_family; // Семейство адресов
    unsigned short int sin_port; // Номер порта
    struct in_addr sin_addr; // IP-адрес
    unsigned char sin_zero[8]; // Дополнение до размера структуры sockaddr
};
```

Здесь поле **sin_family** соответствует полю **sa_family** в **sockaddr**, в **sin_port** записывается номер порта, а в **sin_addr** - IP-адрес хоста. Поле **sin_addr** само является структурой, которая имеет вид:

```
struct in_addr {
    unsigned long s_addr;
};
```

Зачем понадобилось заключать всего одно поле в структуру? Дело в том, что раньше **in_addr** представляла собой объединение (union), содержащее гораздо большее число полей. Сейчас, когда в ней осталось всего одно поле, она продолжает использоваться для обратной совместимости.

Существует два порядка хранения байтов в слове (2 байта) и двойном слове (4 байта). Один из них называется *порядком хоста* (host byte order), другой - *сетевым порядком* (network byte order) хранения байтов (big-endian).

Рассмотрим пример.

4-байтное целое число 0x01020304 будет сохранено в памяти системы big endian следующим образом:

Байт0	Байт1	Байт2	Байт3
0x01	0x02	0x03	0x04

Та же самая величина, которая будет храниться в памяти системы little endian, разместится в противоположном порядке:

Байт0	Байт1	Байт2	Байт3
0x04	0x03	0x02	0x01

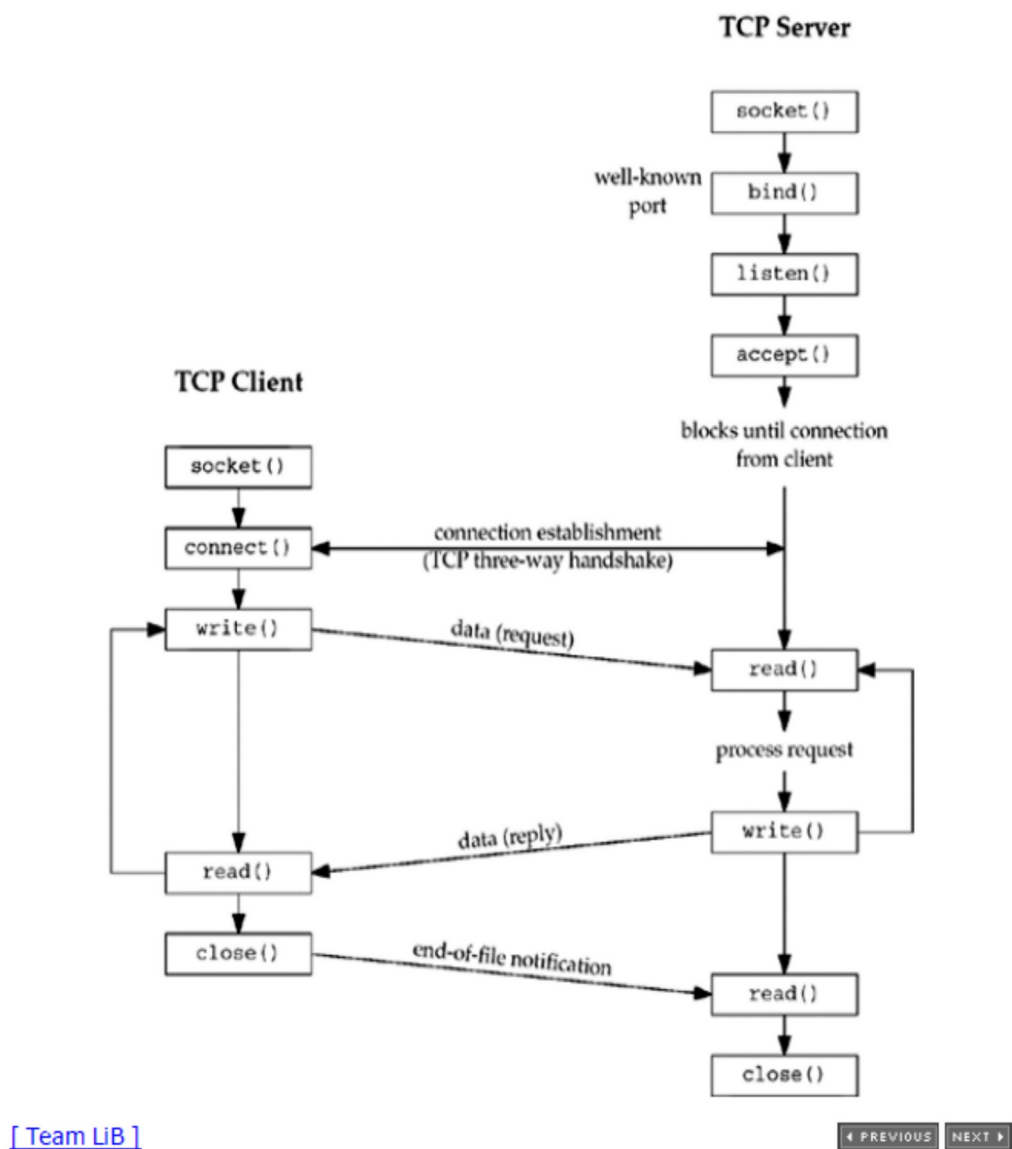
Обычно при программировании можно не обращать внимания на endianness, то есть не важно, как процессор сохранит байты чисел в системе - big endian или little endian; ядро CPU просто загружает данные из памяти и сохраняет данные в память, и представляет данные в Вашей программе уже в правильном виде. Однако, когда нужно обмениваться данными с другой системой, обе системы должны учитывать формат хранения данных в памяти (endianness).

Linux kernel может быть либо big endian, либо little endian, в зависимости от архитектуры, в расчете на которую kernel скомпилировано. Ниже в таблице показан endianness для различных типов архитектур процессоров и протоколов.

Big Endian	Little Endian	Оба варианта
Архитектуры процессоров		
AVR32	Alpha	ARM
FR-V	CRIS	SuperH (sh)
H8300	Blackfin	M32R
PA-RISC	Intel 64	MIPS
S390	IA-32 (x86) MN10300	Xtensa
Motorola 680x0 PowerPC	AT91SAM7	
SPARC		
Протоколы		
TCP/IP	USB	

При указании IP-адреса и номера порта необходимо преобразовать число из порядка хоста в сетевой. Для этого используются функции **htons** (Host To Network Short) и **htonl** (Host To Network Long). Обратное преобразование выполняют функции **ntohs** (Network To Host Short) и **ntohl**.

Взаимодействие параллельных процессов через сокеты выполняется по модели клиент-сервер.



Сетевые сокеты

Сетевые сокеты являются универсальным типом сокетов. Система приложений, предназначенных для работы на одном компьютере, может использовать сетевые сокеты для обмена данными. Использование сетевых сокетов позволяет сделать процесс масштабирования проекта безпроблемным. Однако даже если сокеты используются для обмена данными на одной и той же машине, передаваемые данные должны пройти все уровни сетевого стека, что отрицательно сказывается на быстродействии и нагрузке на систему.

В качестве примера рассматривается комплекс из двух приложений, клиента и сервера, использующих сетевые сокеты для обмена данными. Рассмотрим некоторые фрагменты программы. Прежде всего, надо получить дескриптор сокета:

```

sock_fd = socket(AF_INET, SOCK_STREAM, 0);
if (sock_fd < 0) {
    printf("socket() failed: %d\n", errno);
}
  
```

```
return EXIT_FAILURE;
}
```

В первом параметре функции `socket()` передается константа `AF_INET`, которая указывает, что открываемый сокет должен быть сетевым. Значение второго параметра требует, чтобы сокет был потоковым. Третий параметр равен 0, т.е. протокол выбирается по умолчанию. Затем вызывается функция `bind()`:

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = INADDR_ANY;
serv_addr.sin_port = htons(port);
if (bind(sock_fd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0) {
    printf("bind() failed: %d\n", errno);
    return EXIT_FAILURE;
}
```

Переменная `serv_addr` это - структура типа `sockaddr_in`. Тип `sockaddr_in` специально предназначен для хранения адресов в формате Интернета. Самое главное отличие `sockaddr_in` от `sockaddr_un` – наличие параметра `sin_port`, предназначенного для хранения значения порта. Функция `htons()` переписывает двухбайтовое значение порта так, чтобы порядок байтов соответствовал сетевому. В качестве семейства адресов указывается `AF_INET`, а в качестве самого адреса – специальную константу `INADDR_ANY`. Благодаря этой константе наша программа сервер регистрируется на всех адресах той машины, на которой она выполняется.

Сетевой сервер должен обрабатывать запросы множества клиентов одновременно. При этом в соединениях «точка-точка», например, при использовании потоковых сокетов, для каждого клиента у сервера должен быть открыт отдельный сокет. Для обеспечения такой возможности вызывается функция `listen(2)`, которая переводит сервер в режим ожидания (пассивного) запроса на соединение:

```
listen(sock, 1);
```

Второй параметр `listen()` – максимальное число соединений, которые сервер может обрабатывать параллельно.

Запрос на пассивное открытие соединения означает, что процесс ждет получения запросов на соединение вместо того, чтобы пытаться самому установить его. Часто процесс, сделавший запрос на пассивное открытие, будет принимать запросы на соединение от любого другого процесса. Процессы могут осуществлять пассивные открытия соединений и ждать, пока от других процессов придут соответствующие запросы на активное открытие, а протокол TCP проинформирует их об установлении соединения. Два процесса, сделавшие друг другу одновременно запросы на активное открытие, получают корректное соединение. Гибкость такого подхода становится критичной при поддержке распределенных вычислений, когда компоненты системы взаимодействуют друг с другом асинхронным образом.

Когда осуществляется подбор сокетов для локального запроса пассивного открытия и чужого запроса на активное открытие, то принципиальное значение имеют два случая. В первом случае местное пассивное открытие полностью определяет чужой сокет. При этом подбор должен осуществляться очень аккуратно. Во втором случае во время местного пассивного открытия чужой сокет не указывается. Тогда в принципе может

быть установлено соединение с любых чужих сокетов. Во всех остальных случаях подбор сокетов имеет частичные ограничения.

Если на один и тот же местный сокет осуществлено несколько ждущих пассивных запросов на открытие (записанных в блоки TCB), и осуществляется извне активный запрос на открытие, то чужой активный сокет будет связываться с тем блоком TCB, где было указание именно на этот запросивший соединения сокет. И только если такого блока TCB не существует, выбор партнера осуществляется среди блоков TCB с неопределенным чужим сокетом.

Затем вызывается функция `accept(2)`, которая устанавливает соединение в ответ на запрос клиента и создает копию сокета для того, чтобы исходный сокет мог продолжать прослушивание:

```
newsock_fd = accept(sock_fd, (struct sockaddr *) &cli_addr, &crlen);
if (newsock_fd < 0)
{
    printf("accept() failed: %d\n", errno);
    return EXIT_FAILURE;
}
```

Получив запрос на соединение, функция `accept()` возвращает файловый дескриптор нового сокета, который открыт для обмена данными с клиентом, запросившего соединение. Сервер как бы перенаправляет запрошенное соединение на другой сокет, оставляя сокет `sock` свободным для прослушивания запросов на установку соединения. Второй параметр функции `accept()` содержит сведения об адресе клиента, запросившего соединение, а третий параметр указывает размер второго.

Процедура установки соединения по протоколу TCP использует флаг управления синхронизацией (SYN) и трижды обменивается сообщениями. Такой обмен называется трехвариантным подтверждением.

Соединение инициируется при встрече пришедшего сегмента, несущего флаг синхронизации (SYN), и ждущей его записи в блоке TCB. И сегмент, и запись создаются пришедшими от пользователей запросами на открытие. Соответствие местного и чужого сокетов устанавливается при инициализации соединения. Соединение признается установленным, когда номера очередей синхронизированы в обоих направлениях между сокетами.

Отмена соединения также включает обмен сегментами, несущими на этот раз управляющий флаг FIN.

Так же как и при вызове функции `recvfrom()`, может быть передано значение `NULL` в последнем и предпоследнем параметрах. Для чтения и записи данных сервер использует функции `read()` и `write()`, а для закрытия сокетов, естественно, `close()`.

В программе-клиенте, прежде всего, нужно решить задачу, которой не было при написании сервера, а именно выполнить преобразование доменного имени сервера в его сетевой адрес. Разрешение доменных имен выполняет функция `gethostbyname()`:

```
server = gethostbyname(argv[1]);
if (server == NULL){
    printf("Host not found\n");
    return EXIT_FAILURE;
}
```

Функция получает указатель на строку с интернет-именем сервера (например, www.unix.com или 192.168.1.16) и возвращает указатель на структуру hostent (переменная server), которая содержит имя сервера в приемлемом для дальнейшего использования виде. При этом, если необходимо, выполняется разрешение доменного имени в сетевой адрес.

Далее заполняются поля переменной serv_addr (структуры sockaddr_in) значениями адреса и порта:

```
serv_addr.sin_family = AF_INET;
strncpy((char *)&serv_addr.sin_addr.s_addr,
(char *)server->h_addr, server->h_length);
serv_addr.sin_port = htons(port);
```

Программа-клиент открывает новый сокет с помощью вызова функции socket() и вызывается функция connect(2) для установки соединения:

```
if (connect(sock_fd, &serv_addr, sizeof(serv_addr)) < 0)
{
    printf("connect() failed: %d", errno);
    return EXIT_FAILURE;
}
```

Теперь сокет готов к передаче и приему данных. Программа-клиент, например, может считывать символы, вводимые пользователем в окне терминала. Когда пользователь нажимает <Ввод>, программа передает данные серверу, ждет ответного сообщения сервера и распечатывает его.

По умолчанию функция socket() создает блокирующий сокет. Чтобы сделать его неблокирующим, надо использовать функцию fcntl(2) с флагом O_NONBLOCK:

```
sock_fd = socket(PF_INET, SOCK_STREAM, 0);
fcntl(sock_fd, F_SETFL, O_NONBLOCK);
```

Теперь любой вызов функции read() для сокета sock будет возвращать управление сразу же. Если на входе сокета нет данных для чтения, функция read() вернет значение EAGAIN.

Для проверки состояния неблокирующих сокетов можно использовать мультиплексирование и воспользоваться функциями **select(2)** или **poll()** и их современные версии pselect() и epoll().

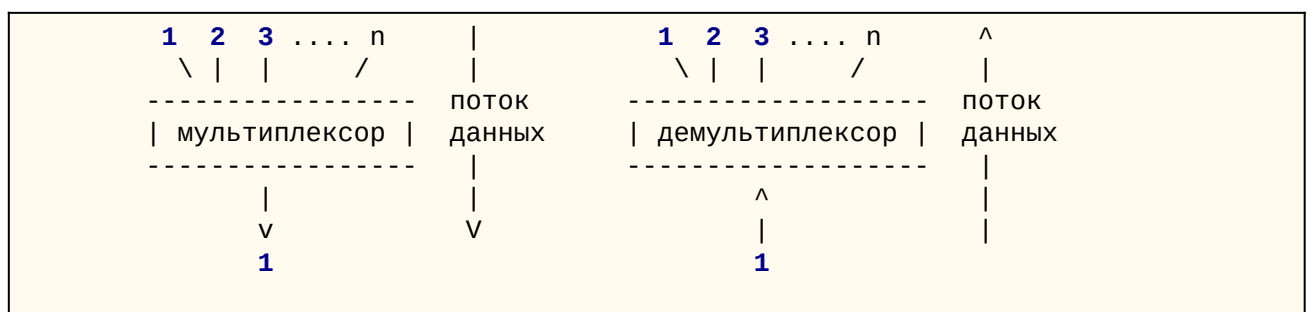


Рис. 2. Мультиплексор $n \times 1$ и демультиплексор $1 \times n$

Функция **select()** (см. Приложение или man pages) способна проверять состояние нескольких дескрипторов сокетов (или файлов) сразу. Первый параметр функции – количество проверяемых дескрипторов. Второй, третий и четвертый параметры функции представляют собой наборы дескрипторов, которые следует проверять, соответственно, на готовность к чтению, записи и на наличие исключительных ситуаций. Сама функция **select()** – блокирующая, она возвращает управление, если хотя бы один из проверяемых сокетов готов к выполнению соответствующей операции. В качестве последнего параметра функции **select()** можно указать интервал времени, по прошествии которого она вернет управление в любом случае. Вызов **select()** для проверки наличия входящих данных на соquete sock может выглядеть так:

```
fd_set set;
struct timeval interval;
FD_SET(sock, &set);
tv.tv_sec = 1;
tv.tv_usec = 500000;
...
select(1, &set, NULL, NULL, &tv);
if (FD_ISSET(sock, &set) {
    // Есть данные для чтения
}
```

Все, что касается функции **select()** теперь объявляется в заголовочном файле `<sys/select.h>` (раньше объявления элементов функции **select()** были разбросаны по файлам `<sys/types.h>`, `<sys/time.h>` и `<stdlib.h>`). В приведенном фрагменте кода `FD_SET` и `FD_ISSET` – макросы, предназначенные для работы с набором дескрипторов `fd_set`.

Задание на лабораторную работу:

Организовать взаимодействие параллельных процессов в сети (ситуация моделируется на одной машине). Сервер обслуживает два типа «клиентов»: производителей и потребителей по классической задаче «производство-потребление».

- Написать приложение по модели клиент-сервер в двух вариантах:
 1. Многопоточный сервер.
 2. Сервер с мультиплексированием (предпочтительно использовать `epoll()`).

Сервер должен обслуживать запросы параллельно выполняемых клиентов. При демонстрации работы программного комплекса необходимо запустить несколько клиентов (не меньше 3) на разных терминалах и продемонстрировать, что сервер обрабатывает обращения каждого клиента, запросившего соединение и посылает клиентам ответные сообщения.

Для защиты программы с `epoll()` или `select()` (macOS) необходимо представить алгоритм работы мультиплексора `epoll` (Пример можно посмотреть в manual page (<https://man7.org/linux/man-pages/man7/epoll.7.html>)).

NAME [top](#)

`epoll` - I/O event notification facility

SYNOPSIS

[top](#)

```
#include <sys/epoll.h>
```

Пример предлагаемого использования (см. manual)

В то время как использование `epoll` имеет ту же семантику, что и `poll(2)`, запускаемого с помощью `edge-triggered`, требует дополнительных разъяснений, чтобы избежать зависаний в цикле событий приложения. В этом примере прослушивающий сокет сервера - это неблокирующий сокет, для которого был вызван системный вызов `listen(2)`.

Функция

```
do_use_fd(events[n].data.fd);
```

Получает в качестве параметра новый готовый файловый дескриптор до тех пор, пока `EAGAIN` не будет возвращен либо `read(2)`, либо `write(2)`.

Управляемое событиями приложение конечного автомата должно после получения `EAGAIN` записать свое текущее состояние, чтобы при следующем вызове `do_use_fd()` оно продолжило `read(2)` или `write(2)` с того места, где оно остановилось ранее.

В примере объявляются переменные:

```
#define MAX_EVENTS 10
struct epoll_event ev, events[MAX_EVENTS];
```

Структура `epoll_event` имеет следующие поля:

```
#include <sys/epoll.h>
```

```
struct epoll_event {
    uint32_t      events; /* Epoll events */
    epoll_data_t  data;   /* User data variable */
};

union epoll_data {
    void          *ptr;
    int            fd; //file descriptor
    uint32_t      u32;
    uint64_t      u64;
};
```

Структура `epoll_event` определяет данные, которые ядро должно сохранить и вернуть, когда соответствующий файловый дескриптор станет готовым.

Литература:

4. Стивенс У., UNIX: Разработка сетевых приложений. - СПб.: Питер, 2004
- 2 W. R. Stevens, S. A. Rago, Advanced Programming in the UNIX® Environment: Second Edition, Addison Wesley Professional, 2005
- 3 Сокеты. Статья из серии "[Программирование для Linux](#)", журнал [Linux Format](#)
• [Андрей Боровский](#), [symmetrica.net](#)
<http://citforum.ru/programming/unix/sockets/>
4. <https://www.rsdn.org/> [Программирование сокетов в Linux](#) Автор: Александр Шаргин
5. RFC1180 Семейство протоколов TCP/IP <https://www.lissyara.su/doc/rfc/rfc1180/>

Приложение 1

Для того, чтобы посмотреть какие порты открыты и кто их открыл достаточно выполнить команду:

`netstat -tulpn`

Если запускать с правами не привилегированного пользователя, то узнаем только какие порты открыты, а кто их открыл узнать можно только с правами суперпользователя.

```
ekaterina@ekaterina-HP-470-G7-Notebook-PC:~/OS/OperatingSystems/lab_26/task_2$ sudo ss -tulpn
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port users:
udp UNCONN 0 0 127.0.0.53%lo:53 0.0.0.0:* users:((("systemd-resolve",pid=813,fd=12))
udp UNCONN 0 0 0.0.0.0:68 0.0.0.0:* users:((("dhclient",pid=2607,fd=6))
udp UNCONN 0 0 0.0.0.0:631 0.0.0.0:* users:((("cups-browsed",pid=1059,fd=7))
udp UNCONN 0 0 0.0.0.0:51555 0.0.0.0:* users:((("firefox",pid=3041,fd=293))
udp UNCONN 0 0 0.0.0.0:43739 0.0.0.0:* users:((("avahi-daemon",pid=1039,fd=14))
udp UNCONN 0 0 0.0.0.0:44673 0.0.0.0:* users:((("firefox",pid=3041,fd=321))
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:* users:((("avahi-daemon",pid=1039,fd=12))
udp UNCONN 0 0 0.0.0.0:47254 0.0.0.0:* users:((("firefox",pid=3041,fd=256))
udp UNCONN 0 0 [::]:34007 [::]:* users:((("avahi-daemon",pid=1039,fd=15))
udp UNCONN 0 0 [::]:5353 [::]:* users:((("avahi-daemon",pid=1039,fd=13))
tcp LISTEN 0 128 127.0.0.53%lo:53 0.0.0.0:* users:((("systemd-resolve",pid=813,fd=13))
tcp LISTEN 0 5 127.0.0.1:631 0.0.0.0:* users:((("cupsd",pid=962,fd=7))
tcp LISTEN 0 5 0.0.0.0:2000 0.0.0.0:* users:((("server.o",pid=4667,fd=3))
tcp LISTEN 0 5 [::]:631 [::]:* users:((("cupsd",pid=962,fd=6))
```

`sudo ss -tulpn`

```
parallels@parallels-Parallels-Virtual-Platform:~/OSBMSTU/sock$ netstat -at
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address Foreign Address State
tcp 0 0 localhost:domain 0.0.0.0:* LISTEN
tcp 0 0 localhost:ipp 0.0.0.0:* LISTEN
tcp 0 0 0.0.0.0:9999 0.0.0.0:* LISTEN
tcp 1 0 10.211.55.6:44924 192.168.0.1:netbios-ssn CLOSE_WAIT
tcp 0 0 10.211.55.6:37960 ec2-52-35-57-239.:https ESTABLISHED
tcp 0 0 10.211.55.6:33790 ec2-34-211-62-63.:https ESTABLISHED
tcp6 0 0 ip6-localhost:ipp [::]:* LISTEN
```

netstat -at